

The 18th Biennial Conference of International Society for Ecological Modelling

GPU Accelerated High Accuracy Surface Modelling

C. Q. Yan^{a,b,c}, T.X. Yue^{a,b*}, G. Zhao^b

^aState Key Laboratory of Resources and Environment Information System, 11A, Datun Road, Anwai, 100101, Beijing 100101

^bInstitute of Geographic Science and Natural Resources Research, 11A, Datun Road, Anwai, 100101, Beijing 100101

^cDepartment of Information Engineering, Shandong University of Science and Technology, 223, Daizong Street, Taian 271019

Abstract

High Accuracy Surface Modelling (HASM) can model surface with high accuracy, while its speed is a major limitation for its application in large scale data. This paper presents HASM-GA, a Graphic Processor Unit (GPU) accelerated High Accuracy Surface Modelling, to construct surface with a significant boost performance. Modern GPU has a highly parallel architecture with hundreds of processors and stream processors, which is a powerful tool for both graphics processing and general purpose computation. We parallel the most computationally-intensive portion of the HASM through NVIDIA's Compute Unified Development Architecture (CUDA) and Quadro 2000 GPU. The results show that one order of magnitude speedup can be achieved by fully using the parallel processing power of the GPU compared with the traditional CPU method.

© 2011 Published by Elsevier B.V. Selection and/or peer-review under responsibility of School of Environment, Beijing Normal University. Open access under [CC BY-NC-ND license](#).

Keywords: GPU; HASM; Surface Modelling; CUDA

1. Introduction

* Corresponding author. Tel.: +86 10 64889841; fax: +86 1064889630.

E-mail address: yue@lreis.ac.cn.

High Accuracy Surface Modelling(HASM) is a newsurface modelling method based on the fundamental theorem of surfaces. Compared with other classical methodsin Geographic Information Systems (GIS), includingthe TLI (Triangulated irregular network with Linear Interpolation), Spline,IDW (Inverse Distance Weighted) and Kriging, it has much higher accuracy [1]. Supported by the sound fundamental theorem of surfaces, it finds a solution for the error problem that had long troubledGIS. Itcan model surface more accurately.However, this method needs to solve a large scale of linear equation group, which will cost too much time and make the method useless for large scale data. To put it into wide application, its speed should be accelerated.

To shorten the computing time, adaptive method of HASM and HASM using multigrid are developed [8, 12]. However, we found it stillcannotmakeHASM a qualitative leap. Although we can accelerate HASM through multi-core CPU or CPU cluster, it would not facilitate the wide application of HASM, for the limited number of cores of single CPU and the unavailable CPU cluster in desktop application.Finally,we turned to**Graphic Processor Unit(GPU)**as a potential solution.

Driven by the insatiable market demand for real-time, high definition 3D graphics, the programmable GPU has evolved from graphic processing into a highly parallel, multithreaded, many-core processor with tremendous computational horsepower and very high memory bandwidth[2]. Parallelizing computationally- and data-intensive tasks through GPU could substantially enhance the performance of models, which have been verified in [3, 4]. However, GPU application in surface modelling remain an area of emerging studying[5]. They have been restricted by the steep learning curve of programming traditional graphics pipeline and specialized hardware. Recently, with the release of CUDA GPU programming method toolkit by NVIDIA, it transformed the way of coding GPU. The CUDA toolkit enables multiple ways to tap into the power of GPU Computing, writing code in CUDA C/C++, OpenCL, DirectCompute, CUDA Fortran and others. Since modern GPU has powerful computation capacity, new uniform architecture, flexibility in terms of programmability and increasing performance, GPU programming for general purpose computing has been enthusiastically received in the area of scientific research. Thus, we can employ powerful computation capacity of GPU to parallel the separable process of HASM to achieve a speedup.

This paper is organized as follows. Section 2 provides mathematical preliminaries of HASM and a brief introduction of CUDA and GPU parallel programming; section 3 presents the algorithms used to solve the huge system of linear equation group; section 4 details the GPU implementation of the algorithm; section 5 shows the results of numerical tests; section 6 lists our final conclusions.

2. HASM and GPU Parallel Computing

2.1. HASM

According to the fundamental theorem of surfaces, a surface is uniquely defined by the first and second fundamental coefficients. The first fundamental coefficients are used to express how the surface inherits the natural inner product of R^3 , in which R^3 is the set of triples (x, y, z) of real numbers [6]. The first fundamental coefficients of surfaces yield information about some geometric properties of the surface, by which we can calculate the lengths of curves, the angles of tangent vectors, the areas of regions, and geodesics on the surfaces. The second fundamental coefficients reflect the local warping of the surface, namely its deviation from the tangent plane at the point under consideration.

If a surface is a graph of a function $z = f(x, y)$ or $r = (x, y, f(x, y))$, the first fundamental coefficients E , F , and G can be formulated as:

$$\begin{cases} E = 1 + f_x^2 \\ G = 1 + f_y^2 \\ F = f_x \cdot f_y \end{cases} \quad (1)$$

The second fundamental coefficients L , M , and N can be formulated as:

$$\begin{cases} L = \frac{f_{xx}}{\sqrt{1+f_x^2+f_y^2}} \\ N = \frac{f_{yy}}{\sqrt{1+f_x^2+f_y^2}} \\ M = \frac{f_{xy}}{\sqrt{1+f_x^2+f_y^2}} \end{cases} \quad (2)$$

The Gauss equation set can be formulated as:

$$\begin{cases} f_{xx} = \Gamma_{11}^1 \cdot f_x + \Gamma_{11}^2 \cdot f_y + L \cdot (E \cdot G - F^2)^{-1/2} \\ f_{yy} = \Gamma_{22}^1 \cdot f_x + \Gamma_{22}^2 \cdot f_y + N \cdot (E \cdot G - F^2)^{-1/2} \end{cases} \quad (3)$$

Where:

$$\begin{aligned} \Gamma_{11}^1 &= \frac{1}{2} (G \cdot E_x - 2F \cdot F_x + F \cdot E_y) \cdot (E \cdot G - F^2)^{-1} \\ \Gamma_{22}^1 &= \frac{1}{2} (2G \cdot F_y - G \cdot G_x + F \cdot G_y) \cdot (E \cdot G - F^2)^{-1} \\ \Gamma_{11}^2 &= \frac{1}{2} (2E \cdot F_x - E \cdot E_y + F \cdot E_x) \cdot (E \cdot G - F^2)^{-1} \\ \Gamma_{22}^2 &= \frac{1}{2} (E \cdot G_y - 2F \cdot F_y + F \cdot G_x) \cdot (E \cdot G - F^2)^{-1} \end{aligned}$$

2.2. CUDA and GPU Parallel Computing

CUDA is a new hardware and software architecture for programming NVIDIA GPU as a data-parallel computing device. The CUDA computing environment provides a standard C like language interface to the NVIDIA GPUs[2]. CUDA is a parallel programming model which not only consist a sequential host program that running on CPU host but also a kernel program that running on parallel GPU device. The host program sets up the data and transfers it to and from the GPU while the kernel program processes the data using a potentially large number of parallel threads. The threads of a kernel are grouped into a grid of thread blocks. The threads of a given block share a local-store and may synchronize via barriers[2]. To optimally utilize the device, programmer should follow 3 overall performance optimization strategies: maximize parallel execution to achieve maximum utilization; optimize memory usage to achieve maximum memory throughput; optimize instruction usage to achieve maximum instruction throughput.

GPU is traditionally used to rasterize the 2D and 3D primitives by graphics libraries like OpenGL and DirectX. With its increasing processing power of transformation and lighting calculation that jobs used to be done on CPU are transferring to GPU. The processing done by GPUs forms a stand rendering pipeline that involves vertex processing and pixel processing. Since these tasks are inherently parallel, GPUs began to use multiple processing cores to accelerate these tasks. As the power of GPUs growing, they gain the capacity to perform tasks at interactive frame rates. To use these capacities efficiently, graphic libraries are modified to allow programmers to write GPU programs called shaders that bypassed the standard rendering pipeline and gave programmers much control over how to process vertexes and pixels. With the increase in parallel processing power and the ability to program the hardware with shaders, people began to try using these capabilities for general-purpose computing. At the early stages, general-purpose computing was not accessible to the average programmer because it demands knowledge about both general graphics programming and creation of shader programs. With the release of NVIDIA'S CUDA architecture in 2006, a major step forward was made that general-purpose GPU computing accessible to a wide audience was taken. The CUDA replaces separate, specialized hardware units previously for vertex and pixel processing respectively with a single (unified) unit which could be programmed to perform either of the processing. Equally, for CUDA enable execution of C/C++ function, or "Kernel" on a target of GPU device, it enables programmers without graphics program experience to write scalable parallel program for GPUs[7].

3. HASM Algorithm With Conjugate Gradient and Preconditioned Conjugate Gradient

3.1. Solution of HASM Equation Set

To solve the partial differential equation group (3), the first and second fundamental coefficients, E, F, G, L, M, and N must be firstly calculated in terms of sampled values when f is simulated. If $\{\bar{f}_{ij}\}$ are the sampled values of f at sampling points, $\{(x_i, y_j)\}$ and $\{\tilde{f}_{ij}\}$ are interpolations in terms of the sampled values $\{\bar{f}_{ij}\}$. Let $f_{ij}^0 = \tilde{f}_{ij}$, and h represents simulation step length, then we can discretize equation set (3) and formulate the $(n+1)$ th iteration of HASMas following.

$$\begin{cases} f_{i+1,j}^{(n+1)} - 2f_{i,j}^{(n+1)} + f_{i-1,j}^{(n+1)} = (\Gamma_{11}^{(n)})_{i,j} \cdot \frac{f_{i+1,j}^{(n)} - f_{i-1,j}^{(n)}}{2} \cdot h + (\Gamma_{11}^{(n)})_{i,j} \cdot \frac{f_{i,j+1}^{(n)} - f_{i,j-1}^{(n)}}{2} \cdot h + \frac{L_{i,j}^{(n)}}{\sqrt{E_{i,j}^{(n)} + G_{i,j}^{(n)} - 1}} \cdot h^2 \\ f_{i,j+1}^{(n+1)} - 2f_{i,j}^{(n+1)} + f_{i,j-1}^{(n+1)} = (\Gamma_{22}^{(n)})_{i,j} \cdot \frac{f_{i+1,j}^{(n)} - f_{i-1,j}^{(n)}}{2} \cdot h + (\Gamma_{22}^{(n)})_{i,j} \cdot \frac{f_{i,j+1}^{(n)} - f_{i,j-1}^{(n)}}{2} \cdot h + \frac{N_{i,j}^{(n)}}{\sqrt{E_{i,j}^{(n)} + G_{i,j}^{(n)} - 1}} \cdot h^2 \end{cases} \quad (4)$$

Where $n \geq 0$; $f_{0,j}^{(0)}$, $f_{i,0}^{(0)}$, $f_{M+1,j}^{(0)}$, $f_{i,M+1}^{(0)}$ are bound conditions [8]. $M+2$ is the lattice number in direction x or y ; $h = \frac{1}{M+1}$.

If the computational domain is normalized to $[0,1] \times [0,1]$, the two equations of basic equation set (4) can be expressed as:

$$AX^{n+1} = D^n \quad (5)$$

$$BX^{n+1} = E^n \quad (6)$$

Where $X^{(n+1)} = (f_{1,1}^{(n+1)}, \dots, f_{1,M}^{(n+1)}, f_{2,1}^{(n+1)}, \dots, f_{2,M}^{(n+1)}, \dots, f_{M-1,1}^{(n+1)}, \dots, f_{M-1,M}^{(n+1)}, f_{M,1}^{(n+1)}, \dots, f_{M,M}^{(n+1)})^T$; A and B , represent coefficient matrixes of the first equation and the second equation in equation set (4) respectively; D^n and E^n are, respectively, the right-hand side vectors of the same equation set (4).

Let $\bar{f}_{ij}$ is value of $z = f(x, y)$ at the p th sampled point (x_i, y_i) in the computational domain, the simulation value should be equal or approximate to the sampling value at this lattice so that a constraint equation set is added to equation set (4). We can get the following expressions

$$\begin{cases} \min \left\| \begin{bmatrix} A \\ B \end{bmatrix} \cdot X^{(n+1)} - \begin{bmatrix} D^{(n)} \\ E^{(n)} \end{bmatrix} \right\| \\ \text{s.t. } S \cdot X^{n+1} = K \end{cases} \quad (7)$$

For sufficiently large λ , expression (7) could be transferred into unconstrained least-squares approximation

$$\begin{bmatrix} A^T & B^T & \lambda \cdot S^T \end{bmatrix} \begin{bmatrix} A \\ B \\ \lambda \cdot S \end{bmatrix} X^{(n+1)} = \begin{bmatrix} A^T & B^T & \lambda \cdot S^T \end{bmatrix} \begin{bmatrix} D^{(n)} \\ E^{(n)} \\ \lambda \cdot K \end{bmatrix} \quad (8)$$

Let $F = \begin{bmatrix} A^T & B^T & \lambda \cdot S^T \end{bmatrix} \begin{bmatrix} A \\ B \\ \lambda \cdot S \end{bmatrix}$, and $T = \begin{bmatrix} A^T & B^T & \lambda \cdot S^T \end{bmatrix} \begin{bmatrix} D^{(n)} \\ E^{(n)} \\ \lambda \cdot K \end{bmatrix}$, expression (8) can be expressed by

$$FX^{(n+1)} = T^n \quad (9)$$

3.2. Conjugate Gradient And Preconditioned Conjugate Gradient Methods

The CG(Conjugate Gradient) is one of the best known iterative methods for solving sparse symmetric positive definite linear systems. The method is flexible, easy to implement and converge(theoretically)in a finite number of steps. While the coefficient matrixes of HASM are symmetric positive definite sparse matrix, CG method is adapted for solving formulation (8). CG algorithm is described as follows[9].

Conjugate Gradient (CG)

$k = 0$: Initialization: $x_0, p_0 = r_0 = b - Ax_0$

$k \geq 0$: While $\|r_k\| / \|r_0\| > \varepsilon$

1. $q_k = Ap_k, \alpha_k = \frac{\|r_k\|^2}{p_k^T q_k}$
2. $x_{k+1} = x_k + \alpha_k p_k, r_{k+1} = r_k - \alpha_k q_k$
3. $\beta_k = \frac{\|r_{k+1}\|^2}{\|r_k\|^2}, p_{k+1} = r_{k+1} + \beta_k p_k$

Each iteration requires one matrix-vector product and two inner products.

Preconditioned Conditioned Gradient (PCG)

The efficiency of CG can be significantly improved with a suitable preconditioning. Preconditioning is a procedure of an application of a transformation, which conditions a given problem into a form that is more suitable for numerical solution. Preconditioning can result in the decrease of the condition number of a matrix, which causes an increased rate of convergence. The idea behind preconditioning is to replace an equation like $Ax = b$ by

$$M^{-1}Ax = M^{-1}b$$

Algorithm is as follows

$k = 0$: Initialization: $x_0, r_0 = b - Ax_0, Mz_0 = r_0, p_0 = z_0$

$k \geq 0$: While $\|r_k\| / \|r_0\| > \varepsilon$

1. $q_k = Ap_k, \alpha_k = \frac{z_k^T r_k}{p_k^T q_k}$
2. $x_{k+1} = x_k + \alpha_k p_k, r_{k+1} = r_k - \alpha_k q_k$
3. $Mz_{k+1} = r_{k+1}$
4. $\beta_k = \frac{z_{k+1}^T r_{k+1}}{z_k^T r_k}, p_{k+1} = r_{k+1} + \beta_k p_k$

Compared with CG, the additional cost is solving one linear system per iteration (to compute z_{k+1} in step 3).

Typically, there is a trade-off in the choice of preconditioner M . Since M must be applied at each step of the iterative linear solver according to the algorithm above, it should have a small cost of applying M . There are several common forms of preconditioning, of which the Jacobi preconditioner is one of the simplest for using the diagonal of the matrix $M = \text{diag}(A)$. Other forms include the Sparse Approximate Inverse preconditioner, Incomplete Cholesky factorization, Incomplete LU factorization, Successive over-relaxation, etc. Since the Jacobi preconditioner is easy to implement efficiently on GPU, we chose it as preconditioner in our PCG.

3.3. GPU Accelerated HASM Algorithm

The GPU Accelerated HASM algorithm can be described as follows. In the first step, we initialized the matrix X with interpolations by Kriging in terms of sampled values. Then, in the second step, the right-hand vectors D^n and E^n of expression (5) and (6) respectively were calculated by finite difference. We got the discretization of equation group (3) by finite difference. After that, we computed the matrix of the left-hand item and the vector of the right-hand item of equation (9). In the following steps, we iteratively solved the huge scale of linear equation set using CG or PCG. Following is the pseudocode description of the algorithm.

HASM-GA

```

Initialize X;//with interpolations according to sampled value S
F=Compute MatrixF(A,B)// compute F according to A and B, A B is already known
For i=1 to outIterNum
    //Compute D and E according to X, by finite difference .h is step length.
    ComputeMatrixDE(X,h,&D,&E);
    ComputeT(D,E,&T);//Compute right-hand vector T in terms of matrix D and E
    Case 0
    CG(F,T,X,&Xnew,InIterNum);
    Case 1
    M=diag(F);
    PCG(F,M,T,X,&Xnew,InIterNum) //PCG iteration choosing Jacobi as preconditioner
End case
    errCheck();//check the error to decide whether break or not
Next

```

4. Implementation of HASM-GA Using CUDA and Corresponding Implementation on CPU

4.1. Details of Implementation

Our implementation can be divided into two sections, of which we implemented the first section with CUDA, and implemented the second section including CG and PCG, mainly employing CUSPARSE and CUBLAS which are libraries of CUDA. CUBLAS is a recent parallel implementation of BLAS, developed by NVIDIA on top of the CUDA programming environment [10]. CUSPARSE is also built by NVIDIA on top of the CUDA programming environment, specially used to process sparse matrix efficiently. The supported compressed storage formats of a sparse matrix by CUSPARSE include Compressed Sparse Row (CSR), Compressed Sparse Column (CSC), Coordinate Format (COO) and Dense Format (DF) [11].

Since our left-hand matrix of expression (9) is a sparse matrix, we can adopt compressed storage format supported by CUSPARSE to reduce the memory cost by saving only non-zero elements and enable the computer process as large scale data as possible. The storage format we adopt is Compressed Sparse Row (CSR) and is described as follows.

In CSR format, a $n \times n$ sparse matrix A , with nnz non-zero entries of A , is stored via three arrays:

$csrValA$ (pointer) : points to the data array of length nnz that holds all non-zero values of A in row-major format.

$csrRowPtrA$ (pointer): points to the integer array of length $m+1$ that holds indices pointing to the array $csrColIndA/csrValA$. For the first m entries, $csrRowPtrA(i)$ contains the index of the first non-zero element in the i th row, while the last entry, $csrRowPtrA(m)$, contains $nnz+csrRowPtrA(0)$.

$csrColIndA$ (pointer): points to the integer array of length nnz that holds the column indices of the corresponding elements in $csrValA$.

The crucial problem in the parallelization of the CG and PCG algorithm on GPU predominantly concerns the inner product and the matrix-vector multiplication. Sparse matrix-vector operations represent the dominant cost in PCG algorithm for solving large-scale linear systems. We implemented sparse matrix-vector using CUSPARSE Library. We did vector operations with CUBLAS. To solve $Mz_{k+1} = r_{k+1}$ efficiently, we used kernels to solve it in a highly parallel way.

4.2. Implementation on CPU

Furthermore, we implement our algorithms on CPU to compare their performance and check out to what extent it can be accelerated.

5. Numerical Experiments and Observations

To compare the performance of CPU implementation and GPU implementation of HASM, we measured the performance across a number of experiments. These simulations were computed on a NVIDIA Quadro 2000 GPU hosted on a Dual-Core Intel Core 2 Duo E8400 running windows XP and CUDA4.0.

In all the algorithms, we started with the same x_k and iterate until $\|r_k\| \|r_0\|^{-1} < 10^{-8}$ for our inner iteration using CG or PCG. We established 5000 as the maximum number of iterations.

5.1. CPU-CG VS GPU-CG

Table 1. HASM-CG-CPU VS HASM-CG-GPU

Scale	HASM-CG-CPU Time(sec)	HASM-CG-GPU Time(sec)	Ratio CG-CPU/ CG-GPU
10*11	0.618339	0.014622	42.28826426
201*201	16.305689	1.830615	8.907219159
970*835	415.454956	40.021626	10.38076154
1024*1125	595.216125	54.681999	10.88505

Table 2. HASM-CG-CPU VS HASM-CG-GPU

Scale	HASM-CG-CPU IterNum	HASM-CG-GPU IterNum	Ratio CG-CPU/ CG-GPU
10*11	95	92	42.28826426
201*201	4250	3862	8.907219159
970*835	5000	5000	10.38076154
1024*1125	5000	5000	10.88505

In table 1, we compared the HASM-CG-CPU implemented on GPU with its CPU counterpart. We can notice that without preconditioning, the HASM-CG-GPU is about 10 times faster than its CPU implementation. Table 2 shows the exact iterative numbers of each experiment. We can obtain a 10 times speedup with processing of large amount data with the maximum iterative number. Additionally, the number of iterations of each experiment is a little different for CPU and GPU implementation because of different supported data precision on CPU and GPU.

5.2. CPU-PCG VS GPU-PCG

Table 3. HASM-PCG-CPU VS HASM-PCG-GPU

Scale	HASM-PCG-CPU Time(sec)	HASM-PCG-GPU Time(sec)	Ratio CPU/GPU
10*11	0.518339	0.011270	45.99281278
201*201	10.390260	1.547023	6.716293164
970*835	465.156982	44.635750	10.42117545
1024*1125	662.741577	61.189999	10.83088

Table 4. HASM-PCG-CPU VSHASM-PCG-GPU

Scale	HASM-PCG-CPUIterNum	HASM-PCG-GPUIterNum	Ratio CPU/GPU
10*11	74	73	45.99281278
201*201	2494	2442	6.716293164
970*835	5000	5000	10.42117545
1024*1125	5000	5000	10.83088

In table 3 and 4, we compared the HASM-PCG-CPU implemented on GPU with its CPU counterpart. We can notice that with preconditioning, the HASM-PCG-GPU is about 10 times faster than its CPU implementation and iterative numbers can be reduced significantly. We also can see that 10 times speedup can be obtained when processing a large scale data with the maximum iterativenumber.

5.3. CPU-CG VSCPU-PCG and CG-GPU VSPCG-GPU

Table 5. HASM-CG-CPU VSHASM-PCG-CPU

Scale	HASM-CG-CPU Time(sec)	HASM-PCG-CPU Time(sec)	Ratio CPU/CPU
10*11	0.618339	0.518339	1.192923936
201*201	16.305689	10.390260	1.569324444
970*835	415.454956	465.156982	0.893149995
1024*1125	595.216125	662.741577	0.898112

In table 5, we compared the HASM-CG-CPU and HASM-PCG-CPU. We can note that, with preconditioning, the HASM-PCG-CPU is faster than HASM-CG-CPU with the experiment of scale 201*201, which shows our preconditioner does its work actually. However, if they both run for the same large number of iterations, HASM-PCG-CPU is slower since it involves solving one more linear system in each iteration. In plus, we can find the similar results from table 6 with their GPU implementation for the same number of iterations.

Table 6. HASM-CG-GPU VSHASM-PCG-GPU

Scale	HASM-CG-GPU Time(sec)	HASM-PCG-GPU Time(sec)	Ratio CPU/GPU
10*11	0.014622	0.011270	1.192923936
201*201	1.830615	1.547023	1.569324444
970*835	40.021626	44.635750	0.893149995
1024*1125	54.681999	61.189999	0.89364275

5.4. HASM-CG-CPU VSHASM-PCG-GPU

Table 7. HASM-CG-CPU VSHASM-PCG-GPU

Scale	HASM-CG-CPU Time(sec)	HASM-PCG-GPU Time(sec)	Ratio
10*11	0.618339	0.011270	54.86592724
201*201	16.305689	1.547023	10.54004304
970*835	415.454956	44.635750	9.307672796
1024*1125	595.216125	61.189999	9.727343271

In table 7, we compared the HASM-CG-CPU and HASM-PCG-GPU implemented. We can observe that with preconditioning, the HASM-PCG-CPU is more than 10 times faster than HASM-CG-CPU with the experiment of scale 201*201. Actually, if they both run for the same large number of iterations, HASM-PCG-GPU is less than 10 times faster than HASM-CG-CPU for solving an additional equation set.

6. Conclusion

We presented a parallel implementation, on GPU, of the High Accuracy Surface Modelling algorithms using CG and PCG. Our experiments show our GPU implementation can accelerate the CPU implementation significantly. For the same large number of iterations, we can get about 10 times speed up. However, for the small scale of data, the GPU implementation cannot show its real advantage over the CPU implementation exactly. On average, the amount of acceleration depends on the scale of the problem and the actual amount of iterations needed to match the error threshold, in general providing more of a 10 times' speedup for bigger and more complex problems.

Acknowledgements

This work is supported by China National Science Fund for Distinguished Young Scholars (40825003), by National High Technology Research and Development Program ("863" Program) (2011AA120306), and by National Natural Science Foundation of China (41023010).

References

- [1] T.-X. Yue, Z.-P. Du, D.-J. Song and Y. Gong. A new method of surface Modelling and its application to DEM construction. *Geomorphology* 2007; **91**: 161-72
- [2] NVIDIA.C. *NVIDIA CUDA C Programming Guide v4*, NVIDIA Corporation, Santa Clara, California.2011;1-101
- [3] J. E. Stone, D. J. Hardy, I. S. Ufimtsev and K. Schulten. GPU-accelerated molecular modeling coming of age. *Journal of Molecular Graphics and Modelling* 2010; **29**: 116-25
- [4] A. J. Kalyanapu, S. Shankar, E. R. Pardyjak, D. R. Judi and S. J. Burian. Assessment of GPU computational enhancement to a 2D flood model. *Environmental Modelling & Software* 2011; **26**: 1009-16
- [5] X. Yang, X. Pi, L. Zeng and S. Li. GPU-based real-time simulation and rendering of unbounded ocean surface. 2005;
- [6] T. X. Yue. *Surface Modeling: High Accuracy and High Speed Methods*, CRC Press.
- [7] E. Wynters. Parallel processing on NVIDIA graphics processing units using CUDA. *Journal of Computing Sciences in Colleges* 2011; **26**: 58-66
- [8] T.-X. Yue, C.-F. Chen and B.-L. Li. An Adaptive Method of High Accuracy Surface Modeling and Its Application to Simulating Elevation Surfaces. *Transactions in GIS* 2010; **14**: 615-30
- [9] R. Helfenstein and J. Koko. Parallel preconditioned conjugate gradient algorithm on GPU. *Journal of Computational and Applied Mathematics* **In Press, Corrected Proof**
- [10] NVIDIA.C. *CUDA Toolkit 4.0 CUBLAS Library*. NVIDIA Corporation, Santa Clara, California.2011;15-82
- [11] NVIDIA.C. *CUDA CUSPARSE Library*. NVIDIA Corporation, Santa Clara, California.2011;5-62
- [12] Tian-Xiang Yue, Yin-Jun Song, Ze-Meng Fan, 2010. The multi-grid method of high accuracy surface modelling and its validation. *Computer & Geoscience*(accepted).